Make them fast and reliable

Databricks pipeline engineer

A5 PySpark performance

A5-L1 Lazy evaluation, and why every count() re-runs the plan

Four questions, one scan. The strong version reads the delivery once, however many checks you add.

Transformation or action?	Transformations (filter, select, join, groupBy().agg) build a plan. Actions (count, collect, show, write) run it.
What happens on the second count() of an uncached DataFrame?	The whole plan runs again from the source.
Many checks, one scan	One df.agg(...) with count, count_if, sum, min, max expressions.
Reusing an expensive result on serverless	Write it to a Delta table. cache() is not supported on serverless compute.

A5-L2 Built-in functions over Python UDFs

The strong version runs in the JVM, survives NULLs and reads as a rule. Functions that return Column expressions give you the reuse that UDFs promised.

Why are Python UDFs slow?	Every value crosses from the JVM to a Python worker and back. The plan shows BatchEvalPython.
Measured cost on 2 million strings	Built-in 0.30 s, pandas UDF 1.16 s, Python UDF 1.82 s (PySpark 4.0.4, local).
What does a Python UDF receive for NULL?	None. s.strip() then raises and fails the job. Built-ins return NULL for NULL.
Reusable logic without a UDF	A Python function that takes and returns Column expressions, built from F.when, F.regexp_replace and friends.

A5-L3 Joins, broadcast, partitions and reading the plan

One broadcast join replaces a collect, a dict and a UDF, keeps the work distributed, and shows up clearly in the plan.

Join strategy for a large table and a small lookup	Broadcast the lookup: F.broadcast(small). The plan shows BroadcastHashJoin.
Default auto-broadcast threshold in Spark 4.0	10 MB (spark.sql.autoBroadcastJoinThreshold). AQE can also switch to broadcast at run time.
repartition versus coalesce	repartition(n) shuffles to n partitions. coalesce(n) merges without a shuffle and only reduces.
How do you confirm a filter reached the file scan?	df.explain(mode="formatted") and look for PushedFilters on the scan.

A6 Data quality

A6-L1 Data contracts with severity levels and structured results

Results are data, severity decides the action, and one run reports every problem.

```python
CheckName = str
Status = Literal["PASS", "WARN", "FAIL", "SKIP"]
Severity = Literal["CRITICAL", "HIGH", "MEDIUM", "LOW"]

@dataclass(frozen=True)
class CheckResult:
    check_name: CheckName
    status: Status
    severity: Severity
    expected: str
    actual: str
```

What does a contract check produce?	A structured record: check_name, status, severity, expected, actual. Never a sentence.
CRITICAL versus WARN	CRITICAL failures stop the pipeline by raising. WARN results are logged and the run continues.
Why evaluate every check before raising?	One run reports every problem, so the supplier fixes everything in one round trip.
Lakeflow Declarative Pipelines expectations	@dp.expect keeps and records, @dp.expect_or_drop drops, @dp.expect_or_fail stops the update.

A6-L2 Statistical checks that survive empty, single-row and zero data

The strong version returns a result for every wave and every subgroup, and says why when a statistic is undefined.

F.stddev on one row, and mean on an empty frame	Both NULL. stddev is the sample standard deviation.
Division by zero on Spark 4.0 with ANSI on	Raises DIVIDE_BY_ZERO, even for DOUBLE. Use F.try_divide or F.when(b > 0, a / b).
What should a check return when a statistic is undefined?	An explicit SKIP result with the reason. Never nothing, and never a crash.
Why is `if cv:` wrong?	A real CV of 0.0 is falsy. Test missing values with `is None`.

A6-L3 Reconcile counts and totals between layers

Silver and gold totals are computed the same way and compared with tolerance. A mismatch becomes a CRITICAL result with numbers, not a print.

Bronze figure that must match silver's row count for a wave	Distinct (respondent_id, wave_id) keys, not raw bronze rows, because bronze keeps every delivery.
Silver-to-gold reconciliation	In-scope silver weight total = sum of gold weighted_base; in-scope silver rows = sum of unweighted_n.
How to compare floating-point totals	math.isclose(a, b, rel_tol=1e-9, abs_tol=1e-6). Never ==.
Severity of a failed reconciliation	CRITICAL. Every figure for the wave is suspect until it passes.

A7 Observability

A7-L1 Structured logging with the logging module, not print

Every line says what, where, which run and which wave. Failures keep their traceback and still fail the job.

```
base = logging.getLogger(__name__)
log = logging.LoggerAdapter(base, {"run_id": run_id, "wave_id": wave_id})
log.info("merged rows=%d", n)
# INFO pipelines.silver run_id=7f3c wave_id=W14 merged rows=1832
```

Create a module logger	`logger = logging.getLogger(__name__)`
Default level threshold with no configuration	WARNING. log.info(...) prints nothing until the entry point sets a level.
Attach run_id and wave_id to every line	logging.LoggerAdapter(logger, {"run_id": ..., "wave_id": ...})
Inside except	log.exception("...") for ERROR plus traceback, then a bare raise.

A7-L2 A pipeline run log table with an explicit schema

The strong record builds on every run, success or failure, and its types never depend on the values in one row.

Why does the reference log_pipeline_run fail on success?	error_message=None is inferred with no type: CANNOT_DETERMINE_TYPE. Declare a StructType.
UTC timestamp in modern Python	datetime.now(timezone.utc). utcnow() is deprecated from 3.12 and naive.
Which ruff rule catches the missing uuid import?	F821, undefined name.
Where do you write the run record so failures are logged too?	In a finally block, after setting status in try and except.

A7-L3 Alert on failures, and on the silence that hides them

Job notifications catch runs that fail. Scheduled queries catch runs that do not happen, or that succeed at doing nothing.

```
email_notifications:
  on_failure:
    - data-oncall@example.org
```

Why must an except block re-raise?	Otherwise the run succeeds and on_failure notifications never fire.
Job-level versus task-level failure notifications	Job-level notifications are not sent for retried task failures. Use task-level ones to hear about every failed attempt.
How do you detect a pipeline that stopped running?	A scheduled query: no SUCCESS in the run log within the expected window.
Where should alerts go?	A group address or channel, with pipeline, wave, run ID and a link.

A8 Incremental processing

A8-L1 Process only the waves you have not processed, correctly

Same filters on both sides, an explicit anti join, a first-run branch, and an idempotent load behind it.

```python
def unprocessed_waves(spark, bronze, silver, tracker_id) -> list[str]:
    source = spark.table(bronze).filter(F.col("tracker_id") ==
  tracker_id).select("wave_id").distinct()
    done = spark.table(silver).filter(F.col("tracker_id") ==
  tracker_id).select("wave_id").distinct()
    return sorted(r.wave_id for r in source.join(done, "wave_id",
  "left_anti").collect())
```

Rule for incremental wave detection	Filter bronze and silver by the same keys, such as tracker_id, then left-anti-join.
Join type for 'in A but not in B'	left_anti.
First run, silver missing	spark.catalog.tableExists(silver) is False: every bronze wave is new.
What does wave-ID detection miss?	Redeliveries of waves silver already holds. Use ingestion timestamps, change data feed or Auto Loader.

A8-L2 Auto Loader for file ingestion into bronze

Auto Loader replaces a fragile processed-files list with a checkpoint that commits together with the Delta write.

```python
(
    spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("cloudFiles.schemaLocation", schema_path)
    .option("header", "true")
    .load(landing_path)
    .select("*", F.col("_metadata.file_path").alias("source_file"))
    .writeStream
    .option("checkpointLocation", checkpoint_path)
    .trigger(availableNow=True)
    .toTable("research_prod.research_bronze.nat_raw")
)
```

What does Auto Loader's checkpoint store?	Discovered files (in RocksDB) and stream progress, so each file is processed exactly once across restarts.
Default schema evolution mode without a supplied schema	addNewColumns: stop with UnknownFieldException, record the new schema, restart.
Incremental at batch cost	.trigger(availableNow=True) on a scheduled job.
Checkpoint rule	One per stream, in durable storage such as a UC volume. Never shared, never deleted casually.

A8-L3 Change data feed, so downstream layers read only what changed

Read after the last version, drop pre-images, keep the newest change per key, and recompute only what the changes touch.

```sql
ALTER TABLE research_prod.research_silver.nat_respondents
SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

```python
changes = (
```

```
    spark.read.option("readChangeFeed", "true")
    .option("startingVersion", last_version + 1)
    .table("research_prod.research_silver.nat_respondents")
)
```

Turn on change data feed	ALTER TABLE t SET TBLPROPERTIES (delta.enableChangeDataFeed = true)
The four _change_type values	insert, update_preimage (old), update_postimage (new), delete.
Two rules for consuming CDF	Drop update_preimage. Keep the newest change per key by _commit_version.
Where does the next run start?	startingVersion = last processed version + 1 (it is inclusive), or a streaming checkpoint.